□ 第一天深度讲义

大模型智能体 (LLM Agents) - Day 1

适用对象: MBA 学员
阅读时间: 约 60 分钟
配合课件: day1-lesson1-8

第 1-2 课时: 大语言模型基础

1.1 什么是大语言模型?

通俗理解

想象一个读过人类所有书籍、文章、代码的"超级读者"。它不是真正理解内容，而是学会了**语言的统计规律**——给定前文，预测下一个最可能出现的词。

输入：`"今天天气真"`
模型预测：`"好"（概率85%）` / `"糟"（概率10%）` / `"热"（概率5%）`

这个简单的"预测下一个词"任务，在足够大的数据和模型规模下，涌现出了惊人的能力：推理、创作、编程、翻译……

技术演进时间线

时间	里程碑	意义
2017	Transformer 论文	注意力机制革命，告别 RNN
2018	GPT-1 (OpenAI)	预训练 + 微调范式
2018	BERT (Google)	双向理解，NLP 基准刷新
2020	GPT-3 (175B)	规模涌现，Few-shot 学习
2022	ChatGPT	对话式 AI 走向大众
2023	GPT-4	多模态，推理能力飞跃
2024	Claude 3/GPT-4o	长上下文，原生多模态
2025	o1/o3/DeepSeek-R1	推理模型，思维链原生
2026	Claude 4.6/GPT-5	Agent 原生，工具调用优化

Transformer 架构（非技术人员版）

不需要理解数学细节，只需知道 Transformer 的三个关键创新：

1. **注意力机制 (Attention)**: 模型可以"关注"输入中任意位置的信息，不受距离限制
2. **并行计算**: 可以同时处理整个序列，训练速度大幅提升
3. **可扩展性**: 架构简洁，容易通过增加参数获得更强能力

   MBA 视角: Transformer 的发明者大多已离开 Google 创业。这项技术创造了万亿美元的新市场，但发明者的直接回报有限——这是一个关于创新价值分配的经典案例。

1.2 Token 经济学

什么是 Token?

Token 是 LLM 处理文本的基本单位，不是字符，也不是单词，而是介于两者之间的"子词"。

```
"Hello, world!" → ["Hello", ",", " world", "!"]  (4 tokens)
"人工智能" → ["人工", "智能"]  (2 tokens, 中文通常1-2字=1token)
```

为什么 Token 重要?

1. **定价依据**: API 按 token 计费
2. **上下文限制**: 每个模型有 token 上限
3. **成本估算**: 1000 tokens ≈ 750 英文单词 ≈ 500 中文字

主流模型定价对比 (2026 年 2 月)

模型	输入价格	输出价格	上下文窗口
GPT-4-Turbo	$10/M	$30/M	128K
GPT-4o	$5/M	$15/M	128K
Claude 3.5 Sonnet	$3/M	$15/M	200K
Claude 4 Opus	$15/M	$75/M	200K
DeepSeek-V2	$0.14/M	$0.28/M	128K
Kimi K2.5	¥12/M	¥12/M	2M

> **成本计算示例**: 一篇 5000 字的商业分析报告 - 输入（含提示词 + 背景资料）: ~10,000 tokens - 输出: ~3,000 tokens - GPT-4-Turbo 成本: $0.10 + $0.09 = **$0.19** - DeepSeek-V2 成本: $0.0014 + $0.0008 = **$0.002**

1.3 Prompt 工程

为什么 Prompt 重要?

同一个模型，不同的提示词可能导致完全不同的输出质量。Prompt 工程是**零成本**提升 AI 效果的最有效方法。

核心技巧

1. 角色设定 (Role Prompting)

```
普通: "分析一下特斯拉的财报"

优化: "你是一位有15年经验的华尔街分析师，专注于新能源汽车行业。
       请从以下角度分析特斯拉2025Q4财报:
       1. 营收增长驱动因素
       2. 毛利率变化原因
       3. 与市场预期的差异
       4. 投资建议"
```

2. 结构化输出

```
请用以下JSON格式输出分析结果:
{
  "summary": "一句话总结",
  "key_metrics": [
    {"name": "指标名", "value": "数值", "yoy_change": "同比变化"}
  ],
  "risks": ["风险1", "风险2"],
  "recommendation": "买入/持有/卖出"
}
```

3. 思维链 (Chain of Thought)

```
直接问："这个创业项目值得投资吗？"

引导推理："请一步步分析这个创业项目：
          第一步：市场规模和增长潜力
          第二步：竞争格局和壁垒
          第三步：团队背景和执行力
          第四步：商业模式和盈利路径
          第五步：综合以上分析，给出投资建议"
```

4. Few-shot 示例

```
任务：判断客户评价的情感倾向

示例1：
评价："物流很快，产品质量不错"
情感：正面

示例2：
评价："等了一周才到，包装还破损"
情感：负面

请判断：
评价："东西一般般，价格倒是便宜"
情感：
```

企业级 Prompt 管理

挑战	解决方案
版本混乱	Prompt 版本控制（类似代码管理）
效果不稳定	A/B 测试 + 评估指标
团队协作	Prompt 模板库 + 最佳实践文档
安全风险	输入过滤 + 输出审核

1.4 企业应用案例

案例 1: Morgan Stanley 的 AI 投研助手

背景: 摩根士丹利需要处理海量研报、新闻、财务数据

方案: - 基于 GPT-4 构建内部知识库 - 整合 10 万 + 研报和分析文档 - 财务顾问可自然语言查询

效果: - 信息检索时间：小时级 → 秒级 - 年节省时间：人均 300+ 小时 - 客户满意度提升 15%

关键成功因素: - 高质量私有数据 - 严格的合规审核 - 渐进式部署（先内部试点）

案例 2: Klarna 的客服 AI

背景: 瑞典金融科技公司，每月数百万客服请求

方案: - 基于 OpenAI 的客服 Agent - 处理退款、查询、投诉等场景 - 人机协作模式

效果（2024 年数据）: - 处理量：相当于 700 名全职客服 - 解决时间：11 分钟 → 2 分钟 - 客户满意度：持平（关键 !）- 年节省成本：$4000 万

争议与反思: - 引发裁员担忧 - "效率提升"vs"就业影响"的平衡 - 适合重复性高、标准化强的场景

第 3-4 课时: Agentic Workflow

2.1 从对话到工作流

单轮对话的局限

```
用户："帮我分析一下竞争对手"
AI:   [给出泛泛而谈的分析]

问题:
- 缺乏具体数据支撑
- 无法获取实时信息
- 分析深度有限
```

工作流的价值

```
用户："帮我分析一下竞争对手"

工作流自动执行:
1. 搜索引擎获取最新新闻
2. 数据库查询历史财务数据
3. 调用分析模板结构化输出
4. 交叉验证关键信息
5. 生成最终报告

输出：有数据支撑、结构清晰、可追溯的分析报告
```

2.2 LangChain 核心概念

什么是 LangChain?

LangChain 是构建 LLM 应用的开发框架，类似于 Web 开发中的 Django/Rails。它提供:

- **标准化抽象**: 统一不同 LLM 的调用方式
- **组件库**: 预置的记忆、检索、工具模块
- **编排能力**: 将多个组件串联成工作流

核心组件

```
                    LangChain 架构


  Models（模型层）
    LLMs: GPT-4, Claude, 通义, etc.
    Embeddings: text-embedding-3, etc.

  Chains（链）
    LLMChain: 简单的提示词→LLM→输出
    SequentialChain: 多步骤串联
    RouterChain: 条件分支
```

```
Memory（记忆）
  ConversationBufferMemory: 完整历史
  ConversationSummaryMemory: 摘要历史
  VectorStoreMemory: 向量检索历史

Retrievers（检索器）
  各类向量数据库集成

Tools（工具）
  搜索、计算、API调用
  自定义工具封装
```

企业选型考量

因素	LangChain	LlamaIndex	自研
开发速度	□ □ □ □ □	□ □ □ □	□ □
灵活性	□ □ □	□ □ □	□ □ □ □ □
可维护性	□ □ □	□ □ □ □	取决于团队
学习曲线	中等	较低	最高
适用场景	通用 Agent	RAG 为主	特殊需求

2.3 RAG: 检索增强生成

为什么需要 RAG?

LLM 的两个根本局限:

1. **知识截止**: 训练数据有时间截止点
2. **幻觉问题**: 可能生成看似合理但错误的内容

RAG 的解决思路: **先检索，再生成**

RAG 工作流程

```
                    RAG 流程


离线阶段（索引构建）


文档      分块      向量化      存入向量数据库

"公司手册.pdf"


[第1章: 公司简介...]
[第2章: 组织架构...]
[第3章: 规章制度...]
```

```
[0.12, -0.34, 0.56, ...]  (向量)
[0.23, 0.45, -0.12, ...]

向量数据库 (Pinecone/Milvus/Chroma)

在线阶段 (查询响应)

用户问题

向量化      相似度搜索      获取相关片段

[问题 + 相关片段]     LLM     答案
```

分块策略 (Chunking)

分块是 RAG 效果的关键，常见策略：

策略	说明	适用场景
固定大小	每 500 字切分	通用场景
句子边界	按句号/段落切分	文章类
语义分块	按语义相似度切分	高质量要求
递归分块	多级粒度	长文档
父子分块	小块检索 + 大块上下文	平衡精度和上下文

向量数据库选型

数据库	类型	特点	适用场景
Pinecone	云服务	托管、易用	快速上手
Milvus	开源	高性能、可扩展	大规模生产
Chroma	开源	轻量、嵌入式	原型开发
Weaviate	开源	多模态支持	图文混合
pgvector	扩展	基于 PostgreSQL	已有 PG 环境

RAG 优化策略

阶段	优化方向	具体方法
索引	分块质量	语义分块、重叠窗口
检索	召回率	混合检索 (关键词 + 向量)、重排序
生成	准确性	提示词优化、Few-shot 示例
评估	持续改进	RAG 评估指标 (Faithfulness/Relevancy)

第 5-6 课时: Agent 架构

3.1 什么是 AI Agent?

定义

AI Agent 是一个**能自主决策、使用工具、完成复杂任务**的 AI 系统。

与传统聊天机器人的区别:

维度	聊天机器人	AI Agent
交互模式	问答式	任务式
决策能力	被动响应	主动规划
工具使用	无/有限	丰富且动态
任务范围	单轮完成	多步骤复杂任务
错误处理	简单重试	反思改进

Agent 的核心循环

```
                    Agent 核心循环


                       感知
                     Perceive


      执行          思考          规划
       Act           Think          Plan


                 反思
                   Reflect


    循环直到任务完成或达到终止条件
```

3.2 ReAct 框架

论文背景

ReAct (Reasoning + Acting) 是 2022 年 Google Research 和 Princeton 提出的 Agent 框架，将**推理**和**行动**交织进行。

核心思想

```
传统方法:
  思考 → 思考 → 思考 → 行动
```

```
ReAct:
  思考 → 行动 → 观察 → 思考 → 行动 → 观察 → ...
```

实际示例

用户任务："帮我查一下苹果公司最新的市值，并与微软比较"

Agent执行过程：

```
Thought 1: 我需要查询苹果公司的当前市值
Action 1: search("Apple Inc market cap 2026")
Observation 1: Apple Inc. (AAPL) has a market cap of $3.2 trillion...

Thought 2: 现在我需要查询微软的市值
Action 2: search("Microsoft market cap 2026")
Observation 2: Microsoft Corporation (MSFT) market cap is $3.4 trillion...

Thought 3: 我已经有了两家公司的数据，可以进行比较
Action 3: 生成比较分析

Final Answer:
截至2026年2月，苹果市值约3.2万亿美元，微软市值约3.4万亿美元。
微软市值超过苹果约2000亿美元，差距约6%...
```

为什么 ReAct 有效?

1. **透明性**: 每一步推理可追溯
2. **可控性**: 可以中断/调整执行
3. **准确性**: 基于实际观察而非臆测
4. **可调试**: 出错时容易定位问题

3.3 反思机制 (Reflection)

为什么需要反思?

Agent 执行过程中可能出错：- 工具调用失败 - 获取的信息不准确 - 推理逻辑有漏洞

反思机制让 Agent 能够**自我评估并修正**。

反思策略

反思策略对比

```
Self-Refine（自我改进）

同一个LLM生成 → 评估 → 改进
优点：简单、成本低
缺点：可能陷入局部最优

Reflexion（反思记忆）
```

```
失败经验 → 提取教训 → 存入记忆 → 下次避免
优点：持续学习、避免重复错误
缺点：需要维护记忆系统

Critic Agent（批评者Agent）

独立Agent专门负责评估和批评
优点：视角独立、更客观
缺点：成本翻倍、需要协调
```

3.4 规划算法

为什么需要规划?

复杂任务需要分解成子任务，按顺序或并行执行。好的规划能够：- 提高执行效率 - 减少无效尝试 - 处理任务依赖

常见规划方法

Tree of Thoughts (ToT)

```
                    问题


        思路A       思路B       思路C


  A1    A2  B1    B2  C1    C2
   ↓         ↓
 评估       评估       （剪枝）
   ↓         ↓
选择最优路径
```

- **思路**: 探索多个可能的推理路径
- **优点**: 找到更优解
- **缺点**: 计算成本高

Graph of Thoughts (GoT)

- 允许思维节点之间形成图结构
- 支持合并、分裂思路
- 更灵活但更复杂

实际选择建议

任务复杂度	推荐方法	说明
简单	直接执行	ReAct 足够
中等	ToT	关键决策点展开
复杂	分层规划	先粗后细
超复杂	人机协作	Agent 提建议，人做决策

任务复杂度	推荐方法	说明

第 7-8 课时: 记忆与工具

4.1 记忆系统

为什么 Agent 需要记忆?

LLM 的根本限制：**上下文窗口有限**

即使是 200K tokens 的上下文，长期对话、大量历史信息仍然无法完全装入。

记忆类型

```
                    Agent 记忆体系


  短期记忆（Working Memory）

  当前对话的上下文
  实现：直接放入prompt
  挑战：超出窗口限制

  长期记忆（Long-term Memory）

  历史对话、用户偏好、学习到的知识
  实现：向量数据库 + 检索
  挑战：检索准确性、存储成本

  情景记忆（Episodic Memory）

  过去执行任务的完整记录
  实现：结构化存储 + 摘要
  用途：从历史中学习、避免重复错误

  语义记忆（Semantic Memory）

  提取的事实、规则、知识
  实现：知识图谱 / 结构化存储
  用途：事实查询、推理依据
```

MemGPT: 记忆管理的创新

MemGPT（2023）提出了一个类比操作系统的记忆管理方案：

```
                MemGPT 架构


  传统方法：
```

```
[固定大小的上下文窗口，满了就截断]

MemGPT:

  Main Context（主上下文 - 类似内存）
  • 系统提示词
  • 当前对话
  • 工作记忆



          LLM自主管理，决定何时
          存入/读取外部存储



  External Storage（外部存储 - 类似硬盘）
  • 历史对话摘要
  • 用户档案
  • 长期记忆
```

核心创新: LLM 自己决定何时"翻页"——读取/写入外部记忆

4.2 工具使用 (Tool Use)

Function Calling

Function Calling 是让 LLM 调用外部函数/API 的标准机制：

```
              Function Calling 流程


1. 定义函数
{
  "name": "get_weather",
  "description": "获取城市天气",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string"}
    }
  }
}

2. 用户请求
"上海今天天气怎么样？"

3. LLM生成函数调用
{"name": "get_weather", "arguments": {"city": "上海"}}
```

```
4. 应用执行函数，返回结果
{"temperature": 22, "condition": "晴"}

5. LLM根据结果生成回复
"上海今天天气晴朗，气温22度，适合外出。"
```

常见工具类型

类型	示例	用途
搜索	Google, Bing, Brave	获取实时信息
计算	计算器, Wolfram Alpha	数学运算
代码	Python 解释器	复杂计算、数据处理
API	天气、股票、地图	特定领域数据
数据库	SQL 查询	结构化数据访问
文件	读写本地文件	文档处理
浏览器	网页访问和操作	Web 交互

工具安全

风险	说明	缓解措施
注入攻击	恶意输入导致非预期调用	输入验证、沙盒执行
权限滥用	Agent 获取过多权限	最小权限原则
数据泄露	敏感信息被发送到外部	数据脱敏、访问控制
成本失控	无限循环调用	调用次数限制、超时

企业工具集成最佳实践

1. **渐进式授权**: 从只读开始，逐步开放写入
2. **审计日志**: 记录所有工具调用
3. **人在回路**: 高风险操作需人工确认
4. **降级策略**: 工具不可用时的备选方案

☐ 延伸阅读

论文

- Attention Is All You Need (Transformer 原文)
- ReAct: Synergizing Reasoning and Acting (Agent 基础)
- MemGPT: Towards LLMs as Operating Systems
- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

书籍

- 《Building LLM Apps》- O'Reilly
- 《Designing Machine Learning Systems》- Chip Huyen

在线课程

- DeepLearning.AI: LangChain 课程
- Coursera: Generative AI with LLMs

第一天讲义完